Assessly Platform

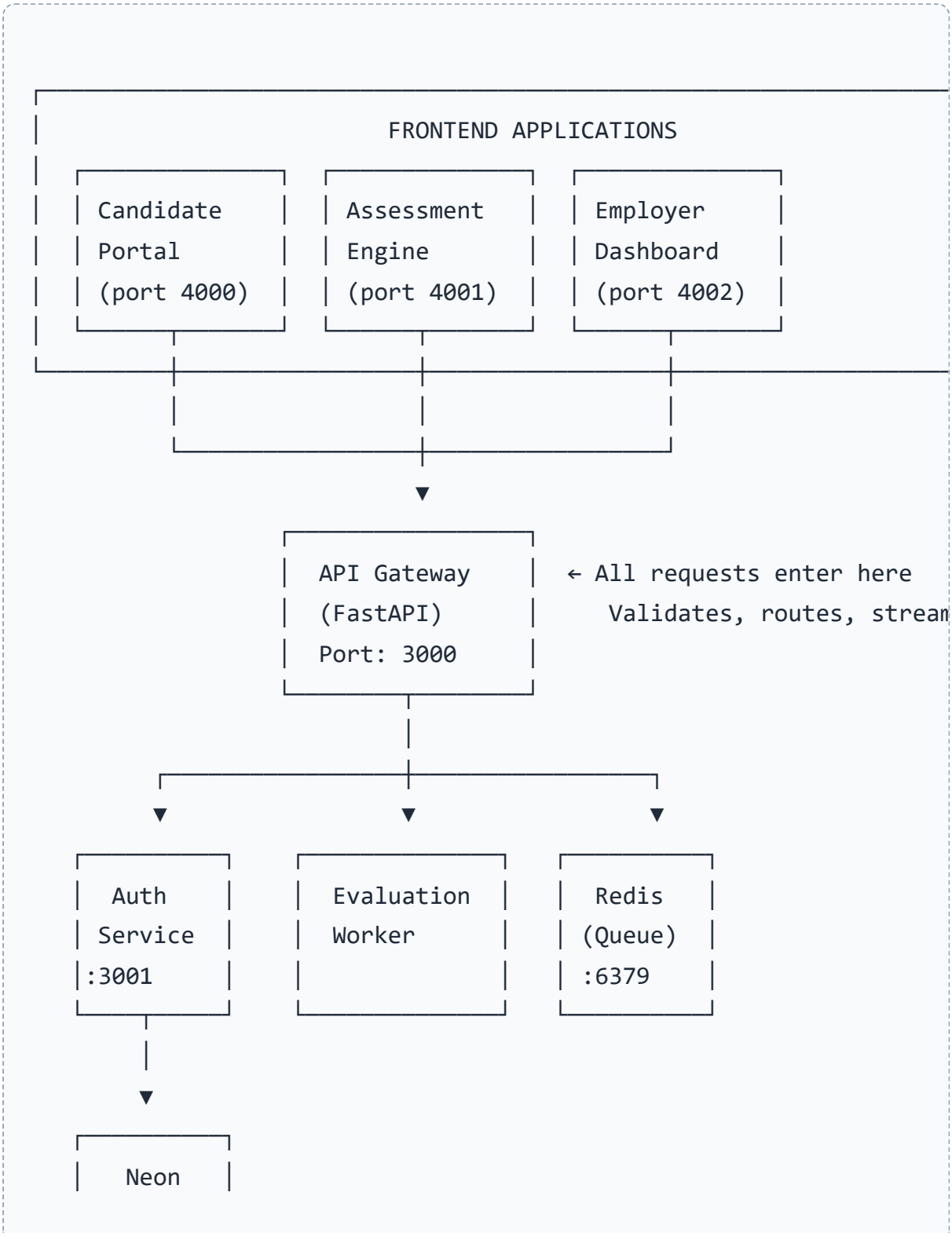
Backend Architecture & Design Decisions Report

Focus: Backend Services, Database Design, Communication Patterns & Security
Project Type: Distributed Candidate Evaluation Platform
Date: May 2026

Table of Contents

1. Architecture Overview (The Big Picture)

The Assessly backend follows a **microservices pattern** — meaning the backend is split into multiple independent services that communicate with each other. Each service has one job and does it well.



```
    | Postgres |
    └──────────┘
```

Simple Analogy: A Restaurant

- **API Gateway** = The reception desk — guests arrive here, orders are taken, food is delivered out
- **Auth Service** = The security guard — checks IDs, issues VIP badges, maintains a blacklist
- **Evaluation Worker** = The kitchen chef — works in the back, prepares meals (grades tests), no guest interaction
- **Redis** = The order ticket rail — carries orders from reception to kitchen, and alerts when food is ready
- **PostgreSQL** = The restaurant's filing cabinet — permanent records of every order, customer, and payment

2. The Three Backend Services

Service	Port	Runtime	One-Line Role
API Gateway	3000	FastAPI + Uvicorn	Main entrance — receives all frontend requests, validates them, routes to correct service
Auth Service	3001	FastAPI + Uvicorn	Security office — login, register, JWT tokens, cross-app authentication
Evaluation Worker	None	Python script	Background grader — scores tests asynchronously, no public URL

3. API Gateway — The Reception Desk

What It Does

- **Receives ALL frontend requests** — every button click, form submission, and API call from the three Next.js apps comes here first
- **Validates requests** using Pydantic models — if data is missing a field or has the wrong type, it rejects with a clear error before any database query runs
- **Routes to Auth** when needed — for protected endpoints, it sends the JWT token to the Auth Service to verify it's valid
- **Enqueues grading jobs** — when a candidate submits a test, it saves answers to PostgreSQL and puts a "grade this" message into Redis
- **Real-time updates via SSE** — keeps a long-lived connection open to the employer dashboard and pushes live events (like "Candidate X just submitted!")
- **Metrics endpoint** — exposes `/metrics` for monitoring latency, request counts, error rates

Why It Exists

> Instead of having frontends talk directly to auth, database, and worker, there is **one front door**. This means:
>
> - Frontend code is simpler — only one base URL to remember
> - CORS is configured in one place
> - Rate limiting, logging, and validation happen centrally
> - The internal services (auth, worker) can be changed without affecting frontend code

Key Endpoints

Endpoint	Method	Auth Required	Purpose

`/api/candidates`	GET	Employer JWT	List candidates with assignments
`/api/assessments`	GET/POST	Employer JWT	List/create assessments
`/api/submissions`	POST	Session JWT	Submit test answers
`/api/questions`	GET	Session JWT	Fetch questions (answers hidden)
`/api/events`	GET	Employer JWT	SSE stream for real-time updates
`/api/scores`	GET	Employer JWT	List evaluated scores
`/api/audit-logs`	GET	Employer JWT	Security audit trail

4. Auth Service — The Security Office

What It Does

- **Login / Register** — checks passwords using `bcrypt` (a slow, secure hashing algorithm), issues JWT tokens
- **JWT Management** — creates access tokens (short-lived, ~15 minutes) and refresh tokens (longer-lived, ~7 days)
- **Cross-App Tokens** — when a candidate clicks "Start Assessment," creates a special one-time token so the Candidate Portal can safely hand the user over to the Assessment Engine
- **Token Verification** — other services (like API Gateway) ask this service: "Is this token valid?"
- **Token Blocklist** — stores revoked tokens in Redis so logged-out users cannot reuse old tokens
- **Audit Logging** — records every login, logout, token mint, and security event

Three Types of JWT Tokens

Token Type	Who Has It	Purpose	Lifetime
Access Token	Candidate / Employer	General API access — calling candidates, assessments, scores	Short (~15 min)
Refresh Token	Candidate / Employer	Get a new access token without re-entering password	Longer (~7 days)
Assessment Session Token	Candidate (during test)	Access test questions and submit answers — separate from main login	Very short (~5 min)

Cross-App Token Flow (Security-Critical)

When a candidate clicks "Start Assessment" in the Candidate Portal:

1. **Portal** (already logged in with access JWT) asks Auth Service for a cross-app token
2. **Auth Service** creates an **opaque random token** (256-bit cryptographically secure random), stores it in Redis with a **120-second expiry**, and cryptographically binds it to the candidate ID + assessment ID using HMAC-SHA256
3. **Portal** redirects the browser to Assessment Engine with `?token=abc123`
4. **Assessment Engine** sends the token to Auth Service
5. **Auth Service** validates the HMAC binding, checks Redis (atomic GET+DELETE for single-use), and returns a fresh assessment session JWT
6. **Assessment Engine** uses that JWT to fetch questions from the API Gateway

> **Why opaque tokens instead of JWTs for cross-app?**
> JWTs are stateless and cannot be instantly revoked. For a **single-use** token, the server needs to remember whether it was used. Opaque tokens + Redis backing is the only correct architectural choice here — it guarantees single-use, instant expiry, and no replay attacks.

5. Evaluation Worker — The Background Grader

What It Does

- **Runs continuously** in the background — no public port, users never talk to it directly
- **Listens to Redis queue** — waits for "grade this test" messages on the `evaluation:queue` list
- **Scores answers** — queries the database for the candidate's responses and the correct answers, compares them
- **Updates results** — writes the score, percentage, correct count, and status back to PostgreSQL
- **Publishes completion** — sends a message to Redis pub/sub so the API Gateway can notify dashboards
- **Handles failures** — retries up to 3 times with exponential backoff, then marks as dead-letter
- **Recovery mode** — periodically scans for submitted tests that never got graded (handles Redis downtime)

Why It's Separate

> Grading is **unpredictable** — 10 candidates might submit at the same time, or a complex assessment might take longer to score. By putting grading in a background worker:
>
> - The API Gateway returns "Submission received!" instantly — the candidate isn't waiting
> - The worker can crash and resume without losing data (jobs stay in Redis)
> - You can run multiple workers in parallel if needed (scale horizontally)

Worker Retry Logic

Scenario	Behavior

Worker crashes mid-grading	Job stays in Redis, picked up by another worker on restart
Database timeout	Retries with exponential backoff (max 3 attempts)
No responses found (empty test)	Marks session as EVALUATED with 0% — no infinite retry loop
Redis pub/sub fails	Score is still saved; dashboard may lag but data is safe
Duplicate submission	Idempotency check prevents double-scoring

6. Shared Layer (Database, Models, Email)

All three services share a common `services/shared/` folder containing:

File	Purpose	Why Shared?
`shared/database.py`	Creates the async SQLAlchemy engine, manages connection pooling, handles DNS fallback, provides `get_db()` dependency	All services connect to the same database with identical settings
`shared/models.py`	Defines all 11 database tables using SQLAlchemy ORM — Users, Assessments, Questions, TestSessions, SessionResponses, etc.	Ensures all services use the exact same table schema and relationships
`shared/email.py`	Sends assignment invitations and result notifications via SMTP	Both API Gateway (when assigning) and Worker (when grading) send emails

Database Connection Setup

The `database.py` file does more than just connect:

- **Connection Pooling** — keeps 5 warm connections, can expand to 15 max (`pool_size=5, max_overflow=10`)
- **Auto-Retry** — if the initial connection fails (DNS hiccup), retries up to 10 times with exponential backoff
- **DNS Fallback** — if your system's DNS can't resolve the Neon database hostname, it falls back to Google DNS (8.8.8.8) and patches Python's socket resolver so future connections work too
- **Pool Pre-Ping** — checks connections before using them, so stale connections are discarded automatically

7. Database Design (PostgreSQL + SQLAlchemy)

Technology Stack

Layer	Technology	Why This One?
Database Server	**Neon PostgreSQL** (cloud)	Serverless auto-scaling; generous free tier; connection pooling built-in; works from anywhere with a URL
Database Driver	**asyncpg**	Fastest PostgreSQL driver for Python; native async support
ORM	**SQLAlchemy 2.0**	Industry standard; full async support; mature migration tool (Alembic); complete query control

The 11 Tables

Table	Purpose	Real-World Analogy
`users`	Candidates & employers (bcrypt passwords, roles, verification)	The school's student and staff register

`assessments`	Test definitions (title, category, duration, pass mark, publish status)	The exam paper template
`questions`	MCQ questions (text, options, correct answer index, points, difficulty)	The actual questions on the paper
`assessment_assignments`	Links candidates to tests they are allowed to take, with due dates	The hall ticket
`test_sessions`	One candidate taking one test (status, score, time taken, timestamps)	The answer sheet
`session_responses`	Each individual answer chosen by the candidate per question	The circles filled in on the answer sheet
`otp_tokens`	One-time password tokens for session verification	A single-use scratch card
`refresh_tokens`	Persistent refresh tokens for session renewal without re-login	A membership renewal coupon
`pending_evaluations`	Backup queue of tests needing grading when Redis is down	A "to be graded" pile on the teacher's desk
`audit_logs`	Complete event log (logins, submissions, admin actions, evaluations)	The security camera footage
`department_benchmarks`	Performance stats by department/category for analytics	The school's annual report card

Smart Indexing (Making Queries Fast)

The database has indexes on columns that are queried frequently:

- `users(email)` — every login looks up by email
- `test_sessions(candidate_id, assessment_id, application_status)` — dashboard filtering
- `session_responses(session_id)` — worker needs all answers for one test
- `audit_logs(user_id, event_type, created_at)` — audit trail searches

8. Communication Patterns

How Services Talk to Each Other

Pattern	Used Between	How It Works
HTTP Requests	Gateway ↔ Auth	Gateway sends the JWT token to Auth Service over HTTP; Auth responds with decoded user info
Redis List (Queue)	Gateway → Worker	Gateway pushes a JSON job to `evaluation:queue`; Worker blocks on `BRPOP` waiting for jobs
Redis Pub/Sub	Worker → Gateway	Worker publishes "grading done" to `scores` channel; Gateway subscribes and forwards via SSE
Redis Key-Value	Auth ↔ Auth	Stores revoked tokens (blocklist) and cross-app tokens with 120-second TTL
PostgreSQL	All ↔ All	All services read/write from the same database — the single source of truth

Server-Sent Events (SSE) — Real-Time Dashboard Updates

The Employer Dashboard shows a "LIVE" badge and updates instantly when candidates submit tests. This uses **Server-Sent Events (SSE)**:

1. Dashboard opens a long-lived HTTP connection to `GET /api/events`
2. Gateway subscribes to the Redis `scores` channel
3. When Worker finishes grading, it publishes a message to Redis
4. Gateway receives the message and writes it to the SSE stream
5. Dashboard receives the event and updates the UI instantly

> **Why SSE instead of WebSockets?**
> SSE is perfect here because:
>
> - The flow is **one-way** (server → dashboard only)
> - Works over standard HTTP — no complex handshake
> - Easy authentication via standard headers
> - Auto-reconnects in the browser if connection drops
>
> WebSockets would be overkill for this use case.

9. Security Architecture

Password Security

- Passwords are hashed with **bcrypt** — a deliberately slow algorithm that makes brute-force attacks impractical
- Plain-text passwords are never stored or logged

JWT Token Scoping

Three different token types exist, each validated separately:

- **Access tokens** — can call general APIs
- **Assessment session tokens** — can only fetch questions and submit answers
- **Refresh tokens** — can only exchange for new access tokens

Correct Answer Hiding

The API Gateway's `/api/questions` endpoint queries the database but **explicitly excludes** the `correct_option` field from the response. Candidates can never see the correct answers through the API.

Audit Trail

Every significant action is logged:

- Login, logout, failed login attempts
- Assessment started, submitted, timed out
- Evaluation completed/failed
- Admin actions (create user, delete user, assign assessment)

Timing-Attack Safety

All token comparisons use `hmac.compare_digest()`, which takes the same amount of time regardless of how many characters match. This prevents attackers from guessing tokens by measuring response times.

10. Data Flow Example: Candidate Takes a Test

```
STEP 1: LOGIN
Candidate Portal → POST /auth/login → Auth Service
Auth Service → checks bcrypt password → returns JWT access + refres

STEP 2: VIEW DASHBOARD
Candidate Portal → GET /api/candidates → API Gateway (with JWT)
API Gateway → queries PostgreSQL → returns assigned assessments

STEP 3: CLICK "START ASSESSMENT"
Candidate Portal → POST /auth/cross-app-token → Auth Service
Auth Service → creates opaque token in Redis (120s TTL, HMAC-bound)
Portal → redirects browser to Assessment Engine ?token=abc123
```

```
STEP 4: LOAD TEST
Assessment Engine → POST /auth/redeem-cross-app → Auth Service
Auth Service → validates token (atomic GET+DELETE) → returns sessic
Engine → GET /api/questions → Gateway (with session JWT)
Gateway → queries PostgreSQL (hiding correct answers) → returns que

STEP 5: SUBMIT ANSWERS
Engine → POST /api/submissions → Gateway
Gateway → saves answers to PostgreSQL
Gateway → pushes job to Redis evaluation queue
Gateway → returns: "Submitted successfully!"

STEP 6: BACKGROUND GRADING (async)
Worker → picks up job from Redis
Worker → queries session_responses + questions from PostgreSQL
Worker → compares answers, computes score
Worker → updates test_session with score + status=EVALUATED
Worker → publishes "EVALUATION_COMPLETED" to Redis pub/sub

STEP 7: REAL-TIME UPDATE
Gateway (subscribed to pub/sub) → receives completion event
Gateway → forwards via SSE to Employer Dashboard
Dashboard → updates candidate score instantly without refresh
```

11. Technology Choices & Why They Were Used

Technology	What It Does	Why It Was Chosen	Alternatives Rejected
FastAPI	Python web framework for APIs	Native async/await; automatic OpenAPI docs; Pydantic validation; extremely fast to develop	Django (too heavy), Flask (no native async), Node.js/Express (different ecosystem)

Python 3.11+	Programming language	Rich async ecosystem; familiar to team; excellent libraries for data processing	Node.js, Go, Java
SQLAlchemy 2.0	Database ORM	Industry standard; full async support; mature migrations via Alembic; complete query control	Prisma (Python support limited), Tortoise ORM (less mature)
asyncpg	PostgreSQL driver	Fastest Python PostgreSQL driver; native async; benchmark-proven	psycopg2 (sync only), aiopg (slower)
Neon PostgreSQL	Cloud database	Serverless auto-scaling; free tier; built-in connection pooling; no local server needed	Local Postgres (needs Docker), AWS RDS (overkill), Supabase (different features)
Redis / Valkey	In-memory data store	Triple duty: task queue + pub/sub messaging + temporary token storage; extremely fast	RabbitMQ (complex), Kafka (overkill), AWS SQS (vendor lock-in)
PyJWT + bcrypt	Token creation + password hashing	bcrypt is the gold standard for password hashing; PyJWT is widely tested	Argon2 (good but less common), plain JWT libraries
SSE	Real-time updates	One-way server→client flow fits dashboard perfectly; no WebSocket handshake needed	WebSockets (overkill for one-way), Long-polling (inefficient)
Pydantic v2	Data validation	Native to FastAPI; automatic request/response	Marshmallow, Cerberus

		validation; clear error messages	
structlog	Structured logging	Outputs JSON logs with context; industry standard for observability; async-safe	Standard logging (unstructured), loguru

12. Project Folder Structure

```
assessly-platform/
├── apps/
│   ├── candidate-portal/       ← Candidate website (Next.js)
│   ├── assessment-engine/      ← Test-taking app (Next.js)
│   └── employer-dashboard/     ← Admin control room (Next.js)
│
├── services/
│   ├── api-gateway/
│   │   ├── main.py             ← All API endpoints, SSE, routing
│   │   ├── config.py           ← Environment settings
│   │   └── test_gateway.py     ← Unit tests
│   │
│   ├── auth-service/
│   │   ├── main.py             ← Login, JWT, cross-app tokens
│   │   ├── config.py           ← JWT secrets, expiry settings
│   │   └── test_auth.py        ← Unit tests
│   │
│   ├── evaluation-worker/
│   │   └── main.py             ← Background scoring pipeline
│   │
│   └── shared/
│       ├── database.py         ← Async SQLAlchemy engine + connect
│       ├── models.py           ← All 11 database table definitions
│       └── email.py            ← SMTP email utilities
```

```
│
├── packages/
│   ├── shared-db/              ← Database migrations (Alembic)
│   ├── shared-types/           ← TypeScript contracts
│   └── shared-ui/              ← Reusable React components
│
├── docs/
│   ├── plan.md                 ← Implementation plan
│   ├── schema-design.md        ← Database design rationale
│   ├── api-design.md           ← API endpoint contracts
│   ├── cross-app-token.md      ← Security mechanism deep-dive
│   ├── pipeline-design.md      ← Worker retry + idempotency
│   └── deployment-design.md    ← Cloud deployment guide
│
├── docker-compose.yml          ← Local orchestration (Redis + serv
├── start-all.py                ← One-command local startup
└── README.md                   ← Full documentation
```

Summary

The Assessly backend is designed as a **distributed, async-first system** with clear separation of concerns:

- **API Gateway** handles all ingress — validation, routing, real-time streams
- **Auth Service** isolates security — JWTs, tokens, password hashing
- **Evaluation Worker** handles background processing — grading, retries, recovery
- **Redis** acts as the nervous system — queue, pub/sub, temporary storage
- **PostgreSQL** is the permanent record — all data, relationships, audit trail

Every technology was chosen for a specific reason: performance (asyncpg, FastAPI), reliability (Redis queue with retries, Neon auto-scaling), security (bcrypt, HMAC-bound tokens, audit logs), and developer velocity (Pydantic, SQLAlchemy, automatic API docs).

> **Key Takeaway:** The architecture separates fast user-facing operations (Gateway) from slow background work (Worker), isolates security (Auth), and

> uses Redis as a lightweight message bus to connect everything without tight coupling.